

面向边缘智能的 AIOS 设计与优化

在 Starry 上承载 RK3588 网球拾取机器人
视觉流水线的实现与系统级优化

赛题编号	proj4
赛题类型	工程型（难度 A）
目标平台	OrangePi 5 Plus / RK3588
参赛队伍	Posad
参赛学校	清华大学
队 员	洪世金、王乙凡、徐子航

摘要

本工作面向边缘智能场景下的具身智能应用，将一台基于 RK3588 的开源网球拾取机器人的操作系统，由基于 C 语言的 Linux 迁移至以 Rust 语言实现的 Starry。该机器人由相机采集图像，经 YOLOv8（一种单阶段实时目标检测网络）模型在片上 NPU（神经网络处理单元，专用于神经网络推理的硬件加速器）上完成目标检测，进而驱动机械臂与底盘完成拾球。

迁移工作分两个层次。前者补齐 Starry 运行该负载所缺的系统能力，包括块设备完成路径的容错、机械臂控制器所用的 USB 串口驱动、底盘电机所用的 PWM 控制与 reboot 系统调用，以及 Starry 此前没有的 RGA（二维图像缩放与色彩转换）和 JPU（JPEG 硬件解码）两个加速器驱动，使同一份可执行文件在 Starry 上得到与 Linux 一致的检测结果。后者建立可测量的剖析口径，在同一份程序上以 Linux 为基线逐阶段定位开销，并以一组优化将端到端推理时延逐步压低，最终在与相机输出一致的输入尺寸下达到 11.75 ms，与 Linux 的 11.4 ms 处于同一水平。

工作过程中形成的若干通用能力已在内核中实现，其中基于硬件性能监控单元的原生 perf 工具与 RGA 驱动已作为合并请求进入主线仓库。

目录

一、 引言	3
二、 支撑机器人运行的系统能力	5
(一) 启动与根文件系统的接入	5
(二) 外设驱动与系统调用	6
(三) NPU 推理链路的接入	6
三、 性能优化	6
(一) 剖析方法与测量工具	7
(二) 性能概览	7
(三) 用户态优化	9
(四) 内核侧的优化支撑与调度展望	12
四、 基础版本与增量贡献	13
五、 面向 AIOS 的 AI 辅助开发方法	14
六、 队员分工	16
七、 开发过程与时间线	16
八、 可复现性与后续工作	16

1 引言

近年来，边缘智能在 AIoT、无人系统与机器人等领域快速发展，并逐步从实验环境走向复杂的真实场景。以基于 LeRobot 框架的自主网球拾取机器人为例，此类应用将高频视觉感知、实时决策与运动控制集成于一台算力受限的设备之上，对系统的时延、抖动与资源利用率均提出了较高要求。

目前此类应用大多构建于 Linux 之上。Linux 凭借完善的驱动与软件生态显著降低了开发门槛，但其面向通用场景的体系结构，在边缘设备上引入了若干并非必要的开销，例如冗余的系统服务、较长的执行路径、较为复杂的调度，以及相对可观的内存占用。在算力受限的条件下，这些开销会反映到机器人的端到端时延与抖动之上。一个针对该场景深度定制的操作系统，因而有机会更充分地释放硬件性能。

Starry 是一个以 Rust 语言实现、构建于 ArceOS 组件之上的操作系统。本工作的目标是上述网球拾取机器人的内核由 Linux 迁移至 Starry，在 RK3588 开发板上实现或补全运行所需的系统调用、文件接口与关键外设驱动，打通基于 NPU 的推理链路，并以 Linux 为基线，在端到端时延、启动速度、推理帧率、资源占用与能耗比等方面开展系统级优化。整体工作沿三个阶段推进，先在开发板上打通推理链路，再补齐机器人本体的感知与执行驱动以完成实际拾球，最后以 Linux 为基线开展性能优化，分别对应本文第二节与第三节。在这一过程中，我们也把 AI 辅助开发本身作为一项方法上的产出加以总结，见第五节。

图 1 概括了本工作的优化历程，从默认配置出发逐步逼近 Linux，各步的具体数据在第三节展开。

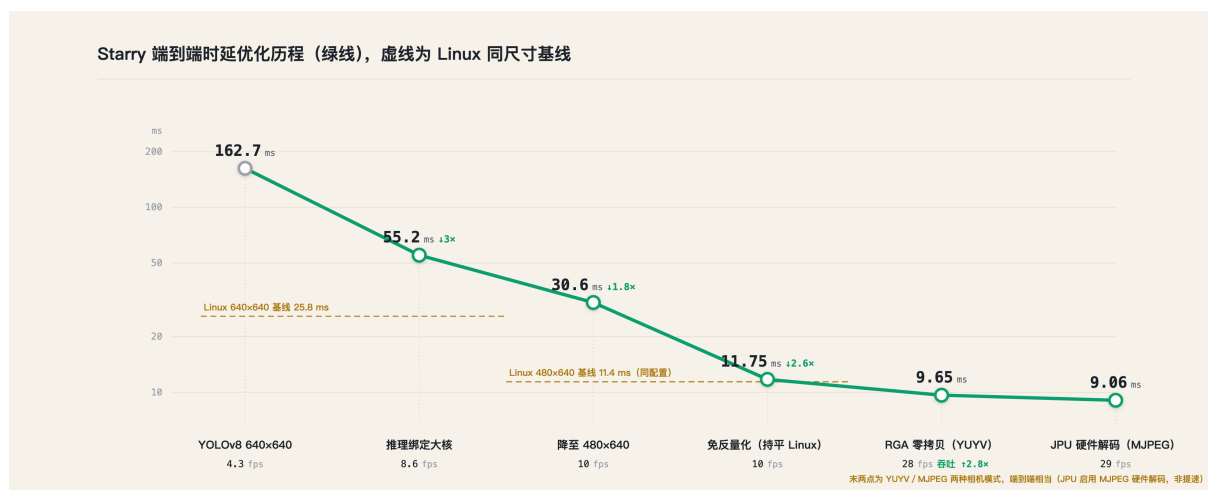


图 1 优化历程示意。前段为 640×640 输入（对照 Linux 25.8 ms），自约 30.6 ms 起改用与相机输出一致的 480×640 输入（对照 Linux 11.4 ms），故曲线后段的下降兼含输入尺寸的变化

为使两侧的比较具有意义，全篇采用同一份遵循 Linux ABI 的可执行文件。该文件在 Linux 与 Starry 上加载相同的推理运行时库 `librknnrt.so` 与相同的模型权重，使观察到的差异尽可能归结于操作系统本身。

测试硬件为 OrangePi 5 Plus 开发板，主控为瑞芯微 RK3588。其处理器采用大小核异构结

构 (ARM big.LITTLE)，含四个 Cortex-A55 小核，主频 1.8GHz，以及四个 Cortex-A76 大核，主频 2.256GHz。片上集成三核 NPU，单核主频 1.0GHz；另有硬件 RGA (Raster Graphic Accelerator，瑞芯微的二维图形加速引擎，可完成图像缩放、色彩空间转换与图层拼接等操作) 与硬件 JPU (此处指 RK3588 的硬件 JPEG 解码单元 VDP720，设备地址 jpegd@fdb90000)。内存为 8GB LPDDR4X。

机器人程序的视觉流水线由若干阶段串联构成。相机的输出有两种格式，YUYV 是未压缩的像素流，只需做色彩空间转换即可得到 RGB；MJPEG 是逐帧 JPEG 压缩的视频流，需先经 JPEG 解码再转换。得到 RGB 后，图像经等比缩放与填充得到模型输入，随后写入 NPU 完成推理，取回输出做后处理得到检测框，最终交由控制环驱动机械臂与底盘完成拾取。整条流程如图 2 所示，从相机采集到机械臂执行构成一个闭环。表 1 给出各阶段的命名与职责，是后文逐阶段数据与剖析所用的统一口径。两侧加载的是同一份模型权重，因此在功能层面具备可比性。所谓 RKNN，是瑞芯微 NPU 所用的运行时与模型格式，常见框架训练出的模型须先经 rknn-toolkit2 工具转换为该格式方可在 NPU 上加载。

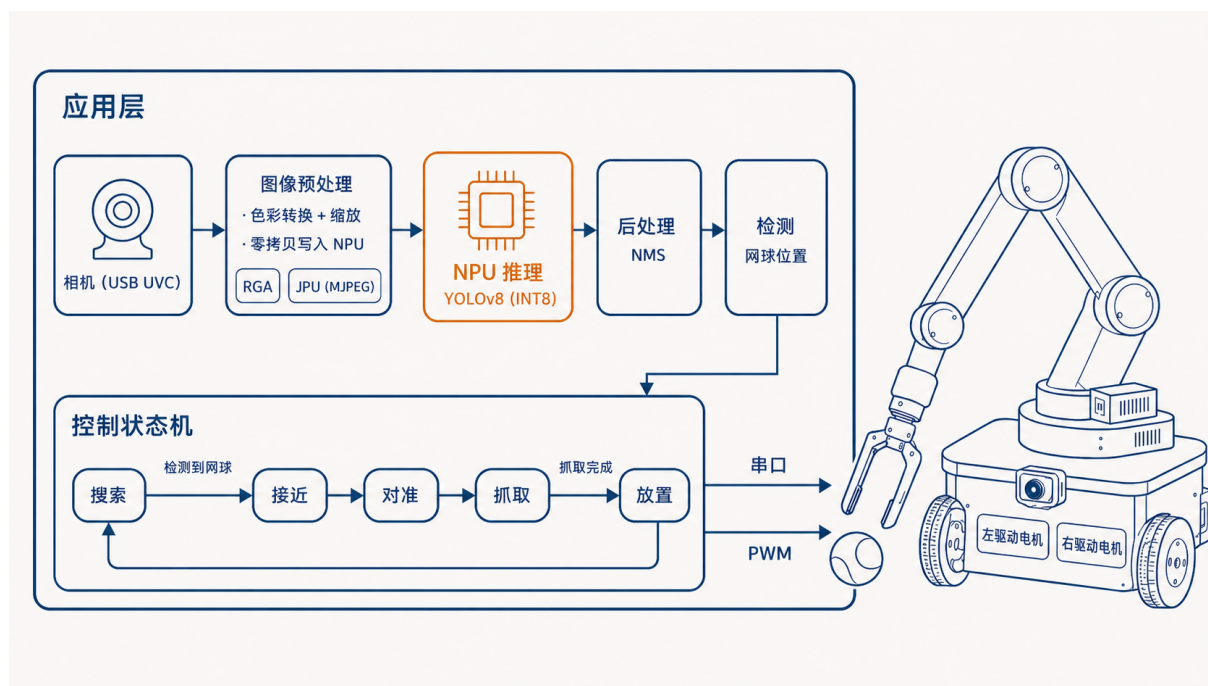


图 2 用户态应用的完整工作流程，相机采集经图像预处理与 NPU 推理得到网球位置，控制状态机据此驱动机械臂与底盘完成拾取

表 1 视觉流水线的阶段划分

阶段	职责
decode	将相机帧由 YUYV 或 MJPEG 解码为 RGB
letterbox	等比缩放并填充至模型输入尺寸
inputs_set	将输入写入 NPU，含必要的缓存维护
run	NPU 完成一次前向推理
outputs_get	取回 NPU 输出，并把其通道分块的特殊排布整理成常规张量（张量重排）
postprocess	把网络输出解码为边框坐标（DFL），再用非极大值抑制（NMS，合并高度重叠的候选框）得到检测框

2 支撑机器人运行的系统能力

将机器人程序在 Starry 上从无到有地跑起来，需要补齐一连串系统能力。本节介绍其中的块设备处理、外设驱动与系统调用，对应评审的第一与第二阶段，与性能优化直接相关的加速器驱动则在第三节介绍。

2.1 启动与根文件系统的接入

将开发板由上电运行至挂载根文件系统，是后续一切工作的前提。Starry 自 SD 卡挂载与 Linux 相同的 ext4 根文件系统，但在注册块设备之后，启动随即停滞。

经排查，相关中断的路由与使能均无误，但该板的 dwmmc 控制器（DesignWare 的 SD/eMMC 主机控制器 IP）在完成一次数据传输后未能稳定拉起完成中断，因而判定问题在控制器一侧。块设备运行时以中断驱动方式等待该完成信号，一旦信号缺失便会一直等待下去，第一次 ext4 读取因此无法返回。

我们在块设备的完成路径上加入了对中断可用性的判定与回退。当一次完成等待在限定时间内未等到中断时，运行时即判定该设备的完成中断当前不可用，转而以轮询方式确认传输是否结束，并对该设备后续的传输沿用这一方式，从而避免缺失的中断令挂载过程停滞。对于中断正常的设备，完成等待在中断到达时立即返回，不引入额外开销。引入这一处理之后，开发板能够稳定挂载根文件系统并进入 shell。

```
//  
//  
match wait_for_completion(&dev, deadline) {  
    Ok(()) => { /* */ }  
    Err(IrqUnavailable) => {  
        dev.use_polling_completion(); //  
        poll_until_complete(&dev);  
    }  
}
```

```
}  
}
```

Listing 1 块设备完成路径的中断可用性判定与回退（节选）

2.2 外设驱动与系统调用

机器人要真正完成拾球，本体的感知与执行须先接齐。相机以 USB 视频设备的形式接入，Starry 已有的 USB 主机栈在 USB3 端口上完成相机的枚举与取流，图像由此进入流水线。机械臂一侧的控制器经一颗 CP210x USB 转串口芯片与主机相连，我们为此实现了 USB 串口驱动，以 `/dev/ttyUSB0` 暴露一个串口设备，机器人程序据此读回机械臂状态并下发控制指令，从而完成定位与抓取。

底盘使用两路直流电机，经 DRV8833 一类 H 桥驱动板接入开发板。每个电机由两根方向控制线决定正转、反转与停止，因而需要四路可输出占空比的 PWM 信号。为使机器人程序在 Linux 与 Starry 上使用同一套控制路径，我们在 Starry 的 `/sys/class/pwm` 中补齐与 Linux 兼容的 sysfs PWM 接口，并在 RK3588 平台侧接入相应的时钟、引脚复用与寄存器配置，使 OrangePi 5 Plus 40 针排针上可用的 PWM 通道能够被导出、设置周期与占空比、启停。

开发初期仅有一块置于远端的开发板，需要在 Linux 与 Starry 之间反复切换以验证同一份程序。为缩短这一回路，我们实现了 reboot 系统调用，使两套系统之间的切换由人工插拔变为一条命令即可完成，明显提高了迭代效率。

2.3 NPU 推理链路的接入

视觉推理依赖片上 NPU。其设备节点与驱动是 StarryOS 已有的部分，我们将其在本负载上调通，以 rknn-toolkit2 将 YOLOv8 模型转换为 RKNN 格式在板上加载。为观察内核侧的开销，我们在该路径上加入了一个可选的计时特性，详见第（一）节，并修复了显存缓冲对象（GEM，图形子系统中用于管理这类缓冲的机制）在销毁与映射环节的若干问题，例如映射时的缺页处理与销毁时的引用计数，二者都会导致下一次推理失败。首次推理在固定图片输入下给出的检测结果与 Linux 完全一致，表明迁移之后模型与运行时的行为正确。

经过本节的工作，机器人从相机感知到底盘运动与机械臂执行的完整回路在 Starry 上跑通。流水线还依赖 RGA 与 JPU 两个硬件加速器，Starry 此前并不支持它们，其驱动由本队实现，因与性能优化直接相关，放在第（四）节介绍。

3 性能优化

推理链路打通之后，工作转入以 Linux 为基线的性能优化，对应评审的第三阶段。本节先建立可测量的剖析口径与工具，再给出两侧的性能概览与差距来源，随后分别从用户态与内核侧展开优化。

3.1 剖析方法与测量工具

可靠的测量是优化的前提。已有的剖析仅提供各阶段的平均值，既无尾部分布，也无内核侧观测与受控消融，其结论停留在某阶段慢若干倍的层面，难以定位开销的物理成因。为此，我们在同一份可执行文件与内核中补齐了观测能力。

应用层在每次推理上记录由相机帧到达主机至检测结果就绪的全链路时延，并给出 p50、p95、p99、p99.9 与 max 各分位，同时以逐帧 CSV 导出供离线分析。冷启动路径单独记录，将从进程启动到首次推理完成的时间拆分为模型初始化、采集初始化、首帧等待与首次推理四段。我们还把拷贝路径与零拷贝路径以同一前端编译为两个二进制，构成输入输出方式的 A/B 对照；所谓零拷贝，是让一段缓冲在各阶段之间不再被复制，而由硬件直接写入下游缓冲。NPU 内部的逐算子耗时以一次性诊断的方式采集，默认关闭，以免扰动 `run` 的主分位（p50、p95 等）数据。内核侧则加入一个可选的计时特性，以无锁原子计数聚合与 NPU 相关 `ioctl` 各阶段的调用次数与耗时，将原本只能推测的缓存同步与提交开销转化为可测量的内核耗时。

为在 CPU 侧获得更细的归因，我们在 Starry 上实现了一套基于硬件性能监控单元（PMU，CPU 内提供周期、指令、缓存与分支等事件计数的部件）的原生 `perf` 工具，它对标 Linux 的 `perf`，读取这些硬件计数器对程序做性能剖析。该工具由按任务采集起步，扩展到对称多处理下的按核采集，再到感知大小核的形态，在两个簇上分别配置各自的 PMU 并对计数器做必要的复用与按核扇出，相关代码已作为合并请求 #1395 提交。该工具在本工作中有三处用途。

- 用每周期指令数（IPC，指单位时钟周期内执行完成的指令条数，越高越接近算力上限，此处并非进程间通信）与缓存计数，判断负载受限于计算还是访存。
- 把各阶段的开销落到周期与指令的尺度上。
- 在大小核两个簇上分别读取计数，量化同一段推理在 A55 与 A76 上的执行差异。

为保证数据可信，干净运行与诊断运行严格分离，前者不开启任何会扰动主分位指标的观测。在合并上游一次较大的内核同步（涉及调度与内存等路径）之后，我们对基线复测了一次，主分位指标全部落在 $\pm 2\%$ 以内。每一项数值结论在成文之前均由多个独立流程对照原始日志逐一核对。

3.2 性能概览

Linux 基线为全篇提供对照参照。在 640×480 的实时采集下，推理绑定至 A76 大核，采样线程隔离于 A55 小核，连续运行 1792 帧无错误，端到端时延平均 25.81 ms，吞吐 29.64 fps。各阶段时延见表 2，其中 `run`，即 NPU 的一次前向推理，约占除解码外的推理各阶段耗时之和的 79%（15.4 与 19.4 ms 之比），是单帧时延的主体。由 NPU 运行时的逐算子剖析进一步可见，该模型为卷积主导，卷积类算子占 NPU 时间的约 72%，因此只有模型层面的改动才能显著缩短 `run`。

表 2 Linux 基线各阶段时延 (640×480 实时, 单位 ms)

阶段	平均	p50	p95	max
端到端	25.81	23.96	33.35	44.30
decode	5.59	4.78	10.18	—
letterbox (RGA)	1.38	1.24	2.20	2.80
inputs_set	0.26	—	0.45	7.50
run (NPU)	15.42	14.82	19.35	28.18
outputs_get	1.30	1.43	1.54	2.24
postprocess	1.06	1.19	1.25	1.86

围绕瓶颈来源的受控消融见表 3。其中以 RGA 承担缩放, 相对 CPU 缩放可将该阶段由 15.82 ms 降至 1.72 ms, 是基线中最显著的单项。NPU 在两核以上无法继续切分该小模型, 逐算子诊断显示其负载几乎集中在一个核上, 第三核没有收益; 其时钟在整个运行中保持 1.0 GHz, 按需调频与锁定最高频两种策略的差异落在噪声以内。此外, 进程的调度时延平均仅 0.021 ms, NPU 每帧读写约 36 MB, 折合约 1.06 GB/s, CPU 缓存缺失率仅 2.3%, 后者由 perf 的缓存计数佐证, 可见基线既不受调度饥饿约束, 也不受内存带宽约束。

表 3 Linux 基线的受控消融

消融项	对照	结果
letterbox 缩放	RGA 硬件 vs CPU	1.72 vs 15.82 ms (9.2×)
NPU 核数	1 / 2 / 3 核	16.66 / 14.87 / 15.23 ms
NPU 调频	ondemand 按需 vs performance 最高频	19.30 vs 18.93 ms (噪声内)
输入输出方式	拷贝 vs 零拷贝	23.83 vs 24.65 ms

在同一份程序、同一份模型的前提下, Starry 默认配置的实时端到端时延 p50 为 162.7 ms, 约为 Linux 的 6.3 倍, 各阶段对照见表 4。内核计时给出了第一个判断, 提交 NPU 作业的内核开销平均仅 0.360 ms, 缓存维护单次仅 0.082 ms, 二者相对 20 ms 量级的推理都微不足道, 差距并不在 NPU 与内核路径之上。在排除采集与解码争用的固定图口径下, run 阶段的 p50 仅为 Linux 的 1.2 至 1.37 倍, 可见 NPU 的计算本身在两侧相近, 实时默认配置下 run 升到数倍, 是同一小核上多项工作相互争用所致。perf 的计数 (较高的 IPC 与很低的缓存缺失率) 进一步显示推理受限于计算而非访存。

表 4 Starry 默认配置与 Linux 的实时逐阶段对照 (p50, 单位 ms)

阶段	Linux	Starry 默认	倍率
letterbox	1.38	75.5	55×
run (NPU)	15.42	71.1	4.6×
outputs_get	1.30	14.3	11×
端到端	25.81	162.7	6.3×

差距集中在单个 A55 小核上的用户态工作，主要是缩放 (letterbox)、输出张量重排 (outputs_get) 与色彩空间转换 (decode) 这几步。其成因在于线程的放置。Starry 的运行队列在派生与唤醒线程时将其置于当前核，且默认没有负载均衡，而机器人程序自身未设置处理器亲和性 (即未把线程绑定到指定的 CPU 核运行)，于是采集、解码、缩放与推理提交全部落在 cpu0 (一个 A55 小核) 之上并相互串行。线程放置造成的串行是差距的一层原因，另一层是缩放等步骤尚未用上 RGA 等硬件加速器而仍由 CPU 承担。后续的优化分别针对这两层展开。

3.3 用户态优化

将推理放置到大核

既然默认放置使全部工作串行于一个 A55 小核，将计算量最大的推理迁往 A76 大核是最直接的一步。需要注意放置的时机，NPU 上下文的创建与 USB 的枚举必须在启动核上完成 (二者的初始化依赖启动核上的特定状态)，否则会挂起，因此亲和性须在模型与相机初始化之后、推理循环之前设置。表 5 给出一次启动内顺次运行的三趟实时对照。

表 5 推理亲和性对端到端时延的影响 (640×480 实时, p50, 单位 ms)。infer_fps 为推理环节的帧率

阶段	默认	绑定 A55	绑定 A76
run (NPU)	71.1	71.2	24.5
letterbox	75.5	74.9	26.4
outputs_get	14.3	14.3	2.7
端到端	162.7	162.8	55.2
infer_fps	4.3	4.6	8.6

核驻留直方图显示，默认与绑定 A55 时推理全部运行于 cpu0，绑定 A76 时全部运行于 cpu4，可见处理器亲和性在 Starry 上得到遵从。将推理绑定至 A76 后，端到端由 162.7 ms 降至 55.2 ms，约为原先的三分之一。采集帧率亦由 10.9 升至 17.7 fps，原因是推理迁出 cpu0 之后，该 A55 核得以服务相机的传输线程。此时相对 Linux 的 25.81 ms 已收敛到约 2.1 倍，残余

差距主要落在 letterbox 的 26.4ms 之上，它仍是 CPU 缩放。

以整数路径取回输出

机器人所用的单类别检测模型，其输出原本以浮点方式取回。NPU 给出的输出是 INT8 整数，取回时需乘以量化尺度还原为浮点，这一步称为反量化。该模型每帧在输出网格上约有四十万个待还原的数值，其中绝大多数对应没有目标的背景。我们改为以整数方式取回原始输出，先在整数域用阈值筛掉这些背景单元，只对少数存活的单元才做反量化。由于反量化是单调变换，在整数域比较置信度与在浮点域等价，结果不受影响。如此每帧的反量化数量由约四十万降到数十，输出获取由约 12ms 降至约 1ms。结合大核放置与 480×640 输入，端到端 p50 达到 11.75ms（见图 3），且在全部 285 张测试图片上检测结果与浮点路径逐一致，没有精度损失。

与之配合的还有两处模型层面的准备。一是把模型按 480×640 的实际输入尺寸导出，使其与相机输出一致，相对 640×640 的方形输入省去了填充并减少约四分之一的检测头计算量。

二是把激活函数由 SiLU 改为 ReLU。SiLU 含有 sigmoid 这类非线性运算，在该 NPU 上不被原生高效支持，而 ReLU 是原生的廉价算子，因此 run 阶段约快两倍半，也更易量化；两者的检测精度相当，见表 6。

表 6 网球检测模型的精度（仿真）。mAP@50 为检测精度，召回为查全率，INT8 余弦相似度衡量量化前后输出的一致性，越接近 1 越好

激活	mAP@50	召回	INT8 余弦相似度
SiLU	0.885	0.919	0.998–1.000
ReLU	0.881	0.887	0.998–1.000

以 RGA 承担解码与缩放

在缩放迁往大核之后，把相机帧由 YUYV 转为 RGB 的色彩空间转换成为 CPU 上最高的一项。RGA 既能承担这一转换，也能承担缩放与填充，还能把结果直接写入 NPU 的输入缓冲，从而把原本分离的解码、缩放与输入设置合并为一步。这里走的是真正的零拷贝路径，也就是让 RGA 直接写入 NPU 的输入缓冲，各阶段之间不再发生内存拷贝。其前提是 NPU 在关闭 IOMMU 的情况下运行。IOMMU 是设备侧的地址翻译部件，为外设提供从设备地址到物理地址的映射，将其关闭后外设直接以物理地址访存，于是 RGA 写出的物理缓冲可被 NPU 原样读取。经此一步，解码 p50 为 0.776ms，输入设置降为零，检测结果完全一致。让这条路径在开发板上真正跑通，依赖于我们为 Starry 实现的 RGA 驱动，其实现见第（四）节。

以 JPU 在硬件上解码 MJPEG

参考机器人的相机以 MJPEG 输出，即逐帧 JPEG 压缩的视频流，须先解码为像素才能交由后续处理。前述各步在 YUYV 模式下由 RGA 直接做色彩转换；改用相机原生的 MJPEG

模式后，这一解码若仍由 CPU 承担，单帧约 25 ms，会重新成为时延的主项。接入 JPU 后，MJPEG 由硬件解码为 NV12，单帧约 1 ms，再交 RGA 转色彩并直写 NPU 的输入缓冲，整条 MJPEG→JPU→RGA→NPU 全程零拷贝。由此在机器人实际使用的 MJPEG 模式下，端到端 p50 为 9.06 ms，约 29 fps，吞吐仍受相机供帧速率限制，与 YUYV 路径的 9.65 ms 基本相当。需说明的是，JPU 在此的作用是为相机原生的 MJPEG 模式提供硬件解码，使该步不致退回 CPU 的约 25 ms，而非相对 YUYV 路径的进一步提速。其驱动实现见第（四）节。

把上述优化叠加起来，差距被逐步压下。模型起初以 640×640 的方形尺寸接收输入，在这一尺寸下，Starry 默认配置的端到端 p50 为 162.7 ms，仅将推理放置到 A76 大核就降至 55.2 ms（见表 5），对应的 Linux 基线为 25.8 ms。把模型输入改为与相机输出一致的 480×640 后，既省去了对方形输入的填充，也减少了检测头的计算量。图 3 给出这一输入尺寸下各步优化的端到端时延，默认为 30.6 ms，叠加整数输出路径与大核放置后降至 11.75 ms，与 Linux 在同一输入尺寸下的 11.4 ms 持平；再以 RGA 零拷贝承担色彩转换降至 9.65 ms，最后由 JPU 在硬件上解码相机原生的 MJPEG，端到端 p50 为 9.06 ms，约 29 fps，受相机供帧速率限制。640×640 与 480×640 是同一模型的两种输入尺寸，前者是优化前的方形输入，后者与相机输出一致、省去了填充，各组数应与对应尺寸下的 Linux 基线相比。

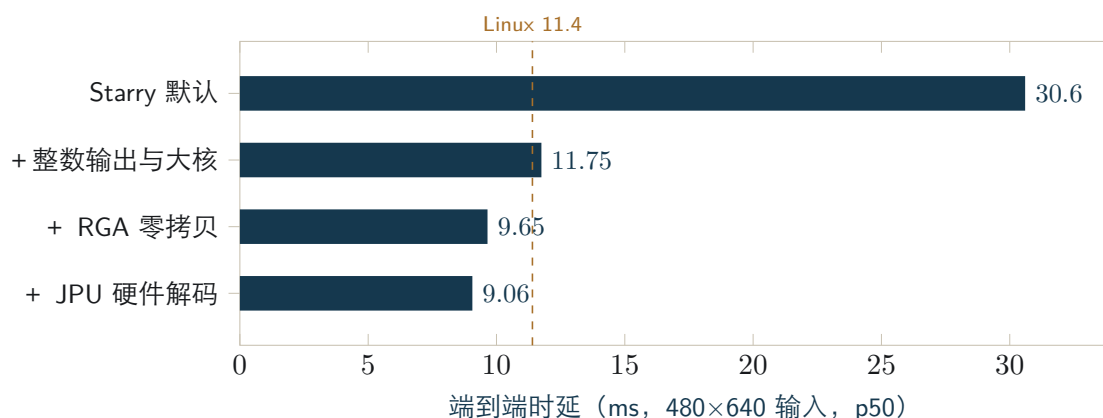


图 3 480×640 输入下各步优化的端到端时延，自默认的 30.6 ms 先降至与 Linux 11.4 ms（赭色虚线）持平的 11.75 ms，再由硬件零拷贝压到约 9 ms（受相机供帧速率限制）。末两条对应 YUYV 与相机原生 MJPEG 两种相机模式，端到端相当，JPU 使 MJPEG 模式得以硬件解码而非相对 YUYV 的提速。横轴只覆盖该输入尺寸，640×640 输入的收敛见表 5

下列三项尝试经测量确认对当前负载没有收益，其原因也指明了各自适用的边界。

- **NPU 张量的零拷贝。**它省去了输入设置的拷贝，却把输出的张量重排从内核侧搬到用户态，在 Starry 的单核上只是把开销换了位置而没有减少，反而把总时延由 217.63 ms 抬高到 266.08 ms。这一手段要在多核并发取走输出的路径下才可能转为收益。
- **第三个 NPU 核。**该模型算子规模较小，逐算子诊断显示其负载几乎集中在一个核上，另两个核接近空闲，在两核之上已无法继续切分。不过闲置的两个核是一块尚未利用的余量，后续可让三核各处理一帧、以三重缓冲提高吞吐，或在空闲核上并行运行另一个模型以扩展机器人的功能，详见第 八 节。

- **锁定 NPU 调频。**它没有带来变化，因为 NPU 在整个运行中已稳定运行在 1.0 GHz。

与 Linux 的整体对照

把优化后的完整流水线放到 Linux 上以同一份程序对照，逐阶段见表 7。在各自的默认调频策略下，Starry 的中位时延更低，但这来自调频策略而非两侧硬件的快慢。Starry 把 NPU、RGA 与 JPU 的时钟直接设为最高且不做动态调频，而 Linux 默认的 `ondemand` 策略在持续负载下会把 NPU 降频。一组固定图的连续推理可看清这一点，Linux 的单次 `run` 由约 4.7 ms 随时间升至约 10.6 ms，而 Starry 始终稳定在约 4.8 ms。也就是说，在同一时钟下两侧的 NPU 推理本就相当，约 4.7 ms，这与 480×640 下 11.75 与 11.4 ms 的持平相互印证；把 Linux 切到最高频策略即可消除中位上的差距。

表 7 完整零拷贝流水线的实时逐阶段对照 (480×640 输入, p50, 单位 ms)。Linux 为其默认的 `ondemand` 按需调频，Starry 不做动态调频

阶段	Linux	Starry
解码 (JPU 与 RGA)	2.11	1.21
run (NPU)	10.89	4.76
outputs_get	0.97	0.73
端到端 (帧到检测)	13.89	9.06

不降频的取舍也有代价，体现在尾部而非中位。Starry 的端到端时延偶有数百毫秒的停顿，最坏一帧达 487 ms，而 Linux 为 22 ms，并因此丢弃约一成的帧；其根源是多核之间的唤醒时延，与第（四）节所述的调度方向直接相关，已提交的空闲唤醒竞态修复是其中一步，尾部表明调度仍有工作要做。此外，Starry 的模型冷启动明显更慢，`.rknn` 加载约 2.2 s，而 Linux 约 0.16 s；常驻内存则反而更省，约 19 MB 对 31 MB。在 60 张固定图的校验下两侧检测结果一致，Starry 为 60/60，Linux 为 59/60，平均置信度 0.880 与 0.875。

3.4 内核侧的优化支撑与调度展望

性能优化中作用于内核侧的部分，关键是让流水线能够把计算从 CPU 交给专用硬件。Starry 此前并不支持 RGA 与 JPU 这两个硬件加速器，让流水线变快首先需要由本队实现它们的内核驱动。

我们为 Starry 实现的 RGA 驱动以 `/dev/rga` 暴露设备节点，应用经其用户态库 `librga` 发起缩放与色彩空间转换等操作。实现它涉及几层工作。底层是 RGA2 引擎的寄存器级驱动，包括开启该模块的时钟、在访问寄存器前解除其软复位、以主模式配置引擎，并以提交、轮询、应答、复位这一状态机驱动每一次操作。其上一层把 `librga` 的请求结构 `rga_req` 按其真实的二进制布局在内核侧镜像出来，再翻译成驱动内部的操作描述，从而无需改动机器人程序即可对接。

把这条路径在开发板上跑通，遇到过三处有代表性的障碍。

- **跨分配器的缓冲导入。**相机与 NPU 的缓冲分属不同的分配器，其句柄无法直接导入，最终改为按物理地址导入才成功。
- **RGA 核的选择。**设备树为 RGA 暴露了三个核，其中一个并不支持所需的操作，最初误锁到该核导致提交失败，修正为按操作选择合适的核。
- **单位旋转矩阵的误判。**librga 在每个请求里都会填入一个表示“不旋转”的单位旋转矩阵，驱动的解析逻辑却把它当成非法的旋转参数拒绝，改为仅在旋转模式非零时才按旋转处理后即解决。

驱动的五项基本操作在开发板上以逐像素校验通过自测，相关代码已作为合并请求 #1388 提交。在 Linux 上的受控对照中，以 RGA 替代 CPU 可将解码由 5.7 ms 降至 0.77 ms；Starry 接入这条路径后，解码 p50 同样为 0.776 ms，缩放也接近归零。

JPU 的驱动以 `/dev/mpp_service` 暴露，遵循瑞芯微多媒体框架 MPP (Media Process Platform) 的用户态接口，为 MJPEG 输入提供硬件解码。其标准解码测试程序 `mpi_dec_test` 在 Starry 上的输出与 Linux 逐字节一致，校验和同为 2712426383；接入机器人程序后，相机的 MJPEG 流亦由 JPU 在硬件上解码，并与 RGA、NPU 共用同一段经 dma-heap 分配的缓冲，构成前述的 MJPEG→JPU→RGA→NPU 零拷贝流水线。两个驱动都采用既有库的真实接口而非自定义的 ioctl 约定，这是它们得以与未经改动的机器人程序对接的关键。

第（一）节所述基于硬件 PMU 的 perf 工具，是另一项在整个优化过程中都用到的内核能力，它让各阶段的开销可以被测量。

线程放置是内核侧仍有空间的一处。第（二）节指出，默认情况下采集、解码与推理都落在同一个小核上相互串行，本工作通过用户态绑核把推理迁往大核解决了当前负载的放置。一个更一般的做法是在内核中实现架构感知的调度，依据各核的算力自动放置线程，使繁重的工作无需手动绑核即可落到大核，并使内核侧的服务线程，例如服务相机传输的 USB 处理线程，也获得合适的放置。我们为此实现了一个感知大小核的负载均衡器原型，它从设备树读取各核的算力权重并据此放置线程，在 QEMU 多核环境下通过了亲和性迁移、抢占与竞态等调度测试，迁移逻辑得到验证。对当前这一单线程且已手动绑核的负载，它不改变运行结果，因此本工作仍采用手动绑核，而将其发展为完整的架构感知调度列为后续方向，详见第八节。

机器人从相机感知到机械臂抓取的完整回路已在 Starry 上跑通，检测结果与 Linux 一致。端到端推理时延在 640×640 输入下由 162.7 ms 收敛至 55.2 ms，在与相机输出一致的 480×640 输入下达到 11.75 ms，与 Linux 处于同一水平。RGA 与 JPU 两个加速器的驱动、基于硬件 PMU 的 perf，以及块设备完成路径的容错等能力都已在内核中实现，可继续支撑 Starry 承载其它边缘智能负载。

4 基础版本与增量贡献

本工作在若干开源项目的基础上展开，相关依赖在首次提交中即予标注。操作系统内核基于 rcore-os/tgoskits 的 dev 分支 (commit 73409e079, 2026-06-22)；机器人应用移植自 pengzechen/aka-

rk3588，我们将其改造为一个 tgoskits 应用；模型转换使用 airockchip/rknn-toolkit2。本队在此基础上的增量贡献见表 8，其中多项已作为合并请求提交至上游仓库 rcore-os/tgoskits。

表 8 本队的增量贡献，PR 均提交至上游仓库 rcore-os/tgoskits

贡献	说明	上游提交
硬件 PMU perf	按任务到大小核的原生 perf	PR #1395，已合并
USB 串口驱动	机械臂控制器的串口通信	PR #1378，已合并
reboot 系统调用	加速 Linux 与 Starry 的切换	PR #1358，已合并
rknpu 整合与缓冲修复	DRM ioctl 整合、GEM 映射修复	PR #1351、#1364，已合并
RGA 驱动	/dev/rga，对齐 librga 接口	PR #1388，开放中
JPU 驱动	/dev/mpp_service，对齐 MPP 接口	待提交
块设备完成路径容错	使开发板得以稳定挂载启动	待提交
PWM 驱动	底盘电机的 sysfs PWM 控制	待提交
profile-rknpu 计时特性	NPU ioctl 各阶段计数	待提交
大小核负载均衡器（原型）	架构感知调度的基础	待提交
剖析工具链与基准应用	同一 ELF 的可测量剖析	随应用提交

5 面向 AIOS 的 AI 辅助开发方法

把一个研究型操作系统从能跑做到与 Linux 同档，涉及大量驱动、系统调用与调优工作。我们在其中系统性地使用了 AI 工具，并围绕一个问题组织整个流程，即怎样让 AI 产出真正可靠的操作系统代码，而不是看似合理却未必正确的代码。我们的做法是把“正确”变成可由机器判定的闸门，让 AI 的每一处改动都先通过闸门才被接受，再以真机闭环与实测数据驱动迭代，而系统架构与最终验收始终在人这一侧。整个过程如图 4 所示。

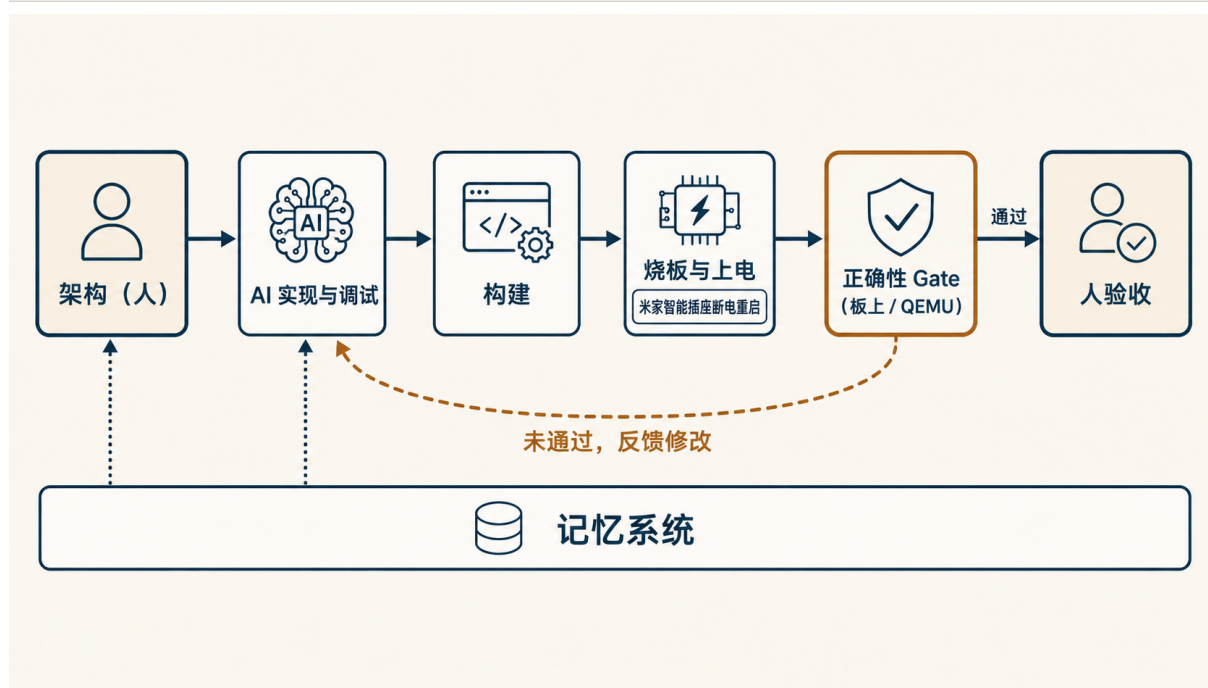


图 4 AI 辅助开发的闭环。人给定规格与架构、并负责最终验收（深色块）；AI 在规格内实现与调试，每次改动须通过板上或 QEMU 的正确性闸门（赭色边框）方可入库，未通过则反馈修改。一只米家智能插座在内核卡死时给开发板断电重启，reboot 系统调用用于切换 Linux 与 Starry，记忆系统在多次会话间承接决策与上下文。

这一方法由几条相互支撑的原则构成。

可机器判定的正确性闸门。操作系统代码的好坏首先在于正确。对驱动与加速器这类工作，我们把正确性落到可逐位核对的基准上，包括 RGA 操作的逐像素 CRC、JPU 解码的校验和、整数输出在 285 张图片上与浮点路径逐一一致、量化模型的余弦相似度，以及 QEMU 与板上的测试用例。AI 的改动只有通过对应闸门才被接受，仅凭看似正确并不能通过。

真机闭环。许多内核问题只在真实硬件上才暴露。我们让迭代直接跑在开发板上，一只米家智能插座负责在内核卡死时给板子断电重启，使一次挂起不至于堵住整条流程；reboot 系统调用（见表 8，上游 PR #1358）则用于在 Linux 与 Starry 之间切换，以同一份可执行文件作对照。由此迭代得以在真机上自动进行。

测量驱动。优化以第（一）节的剖析口径为准绳，针对实测的逐阶段数据下手而非凭推断。过程中若干基于推断的结论都被实测数据修正，“在同一份可执行文件上超过 Linux”这一设想也在测量面前被否定。

对抗式验证。对每一个性能或正确性主张，我们以独立的复核流程主动尝试反驳，只有经得起反驳的结论才写入结果，从而过滤掉 AI 容易产生的过度断言。

记忆与规格先行。架构与目标由队员以规格的形式给定，一个记忆系统在多次会话之间承接决策与上下文，AI 在规格之内执行，队员审阅。

人掌控架构与验收。上述分工的核心是，AI 加速的是规格之内的实现与调试，而系统如何设计、什么算正确、改动是否采纳，都由队员决定。这也使队员能够在无 AI 辅助的条件下解释

关键设计与代码，并在现场完成定位与修改。

按赛事要求，本工作所用的 AI 工具与大模型为 Claude Code（Claude Opus 4.8 与 Sonnet 4.6），以及用于生成示意图的 ChatGPT（GPT-4o 图像生成），使用场景包括资料检索、代码生成、调试定位、日志整理与文档撰写；与 AI 的交互记录予以保留，按工作单元归并的脱敏摘要见提交仓库的 docs/AI .md。

6 队员分工

本工作由三名队员分工承担，见表 9。

表 9 队员分工

成员	负责模块与交付物
洪世金	剖析方法与基线、用户态优化、RGA 与 JPU 驱动、硬件 PMU perf、块设备完成路径容错、报告撰写
王乙凡	USB 串口驱动、reboot 系统调用、PWM 驱动
徐子航	模型训练与数据集准备、系统测试与现场验证

7 开发过程与时间线

本工作的主要开发节点见表 10。

表 10 主要开发节点

阶段	主要工作
剖析口径搭建	在同一份程序与内核中补齐观测能力
Linux 基线	在开发板上完成基线的深度剖析
Starry 使能	块设备完成路径容错，首次跑通 NPU 推理
本体接入	USB 串口、reboot 与 PWM 接齐，相机取流，完成拾球
用户态优化	大核放置、整数输出、RGA 零拷贝
内核能力	加速器驱动、硬件 perf 与负载均衡器原型

8 可复现性与后续工作

本工作的环境与复现步骤已整理留存。开发板经直连网络与串口接入主机，基准程序在板上原生构建，Starry 内核在开启相应特性后经 U-Boot 串口引导启动，在无开发板时则经容器中的

QEMU 运行评测，剖析数据以纯标准库渲染为图。完整说明见随附的环境文档。

赛题所关注的性能维度不止单帧时延，还包括启动速度、内存占用与能耗比等。Starry 作为一个以 Rust 实现的精简内核，其常驻内存更省，但模型的冷启动路径明显更慢，整机能耗尚未测量，这几项的实测对照见表 11；缩短冷启动与采集整机能耗均列为后续优化目标。

表 11 多维度指标的 Starry 与 Linux 对照

维度	Linux	Starry	说明
常驻内存 (MB)	31	19	无冗余系统服务，约省三分之一
启动至首次推理 (s)	0.8	3.6	Starry 的模型加载更慢，待优化
整机功耗	—	—	待板上功耗采集

其余可推进的方向亦较清晰。第（四）节所述的架构感知调度是其中之一，它将依据各核算力自动放置用户态与内核态的线程，免去手动绑核。NPU 的三核可独立工作，而当前的串行点在于设备锁在整个推理期间被持有，一个以每核队列与中断组织的并发完成路径，是进一步提升推理吞吐的结构前提。此外，多核唤醒时延造成的端到端尾部停顿有待随上述调度工作一并收敛；块设备完成中断的底层成因也有待进一步分析，当前的容错处理已能保证稳定挂载。

在机器人应用层面，我们还规划了若干方向。

- 引入 ACT 策略（动作分块的模仿学习策略），让机器人学习并执行更复杂的动作；
- 加入语音输入，使任务可以由语音指令触发；
- 设计更稳健的抓取策略，把抓球的成功率做到接近百分之百；
- 支持里程计，让机器人始终记得回收桶的位置，从而把球准确放入。